

Overview:

This guide outlines the steps for the technical team to implement a subscription system where users subscribe via a landing page, send an SMS, and a postback is made to notify the marketer. The system includes:

- A landing page for user interaction.
- Backend logic to handle user data (phone number, clickId).
- Web-to-SMS functionality.
- Postback to notify the marketer after SMS confirmation.

The following sections cover each implementation detail.

System Architecture:

1. **Landing Page (Frontend - React/HTML):** Collects the user's phone number and sends it to the backend.
2. **Backend (API):** Stores the phone number and clickId in a database.
3. **Web-to-SMS:** Opens the user's messaging app with a predefined message ("OK").
4. **Telco Callback Validation:** Validates the SMS and confirms the subscription.
5. **Marketer Postback:** Sends a POST request to notify the marketer after successful subscription.

1. Database Table Setup

The system uses a database table called `tbl_transaction` to store the user's subscription data. Below is the SQL schema for the table:

```
CREATE TABLE tbl_transaction (  
  id VARCHAR(36) PRIMARY KEY,
```

```

productUuid VARCHAR(100),
extRefId VARCHAR(100),
msisdn VARCHAR(15) NOT NULL,
shortCode VARCHAR(10),
clickId VARCHAR(50),
operation ENUM('SUBSCRIBE', 'UNSUBSCRIBE', 'RENEW') NOT NULL,
status ENUM('PENDING', 'SUCCESS', 'FAILED') NOT NULL,
stage ENUM('INITIATED', 'CONFIRMED', 'COMPLETED') NOT NULL,
amount FLOAT,
additional_data TEXT,
description TEXT,
createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
createdBy VARCHAR(50),
lastUpdatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
lastUpdatedBy VARCHAR(50),
trackerId VARCHAR(100)
);

```

2. Landing Page (Frontend)

The landing page collects the user's phone number and submits it, along with the clickId, to the backend. It also triggers the Web-to-SMS functionality.

Key Features:

- **Input Field:** For entering Ethio Telecom mobile number.
- **Submit Button:** Saves the phone number and opens the SMS app with a predefined message.
- **URL Parameters:** Retrieves the clickId from the URL.
- **SMS Pre-fill:** Opens the SMS app with "OK" in the message body.

Sample React Code for Landing Page:

```
"use client";
```

```
import React, { useState, useEffect } from "react";
```

```
import Head from "next/head";
```

```
import styles from "../page.module.css";
```

```
export default function Home() {
```

```
  const [phoneNumber, setPhoneNumber] = useState("");
```

```
  const [clickId, setClickId] = useState("");
```

```
  const [shortCode, setShortCode] = useState("0000");
```

```
  const handlePhoneNumberChange = (e) => {
```

```
    setPhoneNumber(e.target.value);
```

```
};
```

```
  const handleSubmit = (e) => {
```

```

e.preventDefault();

const apiEndpoint = "http://yourdomain.com/api/transaction";

const payload = {
  subscriberId: `251${phoneNumber.slice(-9)}`,
  clickId: clickId,
  shortCode: shortCode,
};

fetch(apiEndpoint, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(payload),
})
  .then(response => response.json())
  .then(data => {
    if (data.status === "success") {
      const message = "OK";

      window.location.href =
        `sms:${shortCode}?body=${encodeURIComponent(message)}`;
    } else {
      console.log("Subscription not activated");
    }
  })

```

```
  })  
  .catch(error => console.error("Error:", error));  
};
```

```
useEffect(() => {  
  const urlParams = new URLSearchParams(window.location.search);  
  const click_id = urlParams.get("click_id");  
  setClickId(click_id || "");  
}, []);
```

```
return (  
  <div className={styles.page}>  
    <Head>  
      <title>Subscription Page</title>  
    </Head>  
    <form onSubmit={handleSubmit}>  
      <input  
        type="tel"  
        placeholder="Enter Ethio Telecom Number"  
        value={phoneNumber}  
        onChange={handlePhoneNumberChange}  
        pattern="^09[0-9]{8}$"
```

```
        required
      />

      <button type="submit">Subscribe</button>

    </form>
  </div>

  );
}
```

3. Backend API for Saving Transactions

The backend API receives the phone number and clickId from the frontend, saves it to the database, and returns a success or failure response. Here's an example of how to handle this:

POST Endpoint to Save Transaction:

- **Endpoint:** /api/transaction
- **Method:** POST
- **Payload:**

```
{
  "subscriberId": "251912345678",
  "clickId": "abc123",
  "shortCode": "0000"
}
```

Sample Node.js (Express) API:

```

const express = require('express');

const app = express();

app.use(express.json());

app.post('/api/transaction', (req, res) => {

  const { subscriberId, clickId, shortCode } = req.body;

  // Simulate saving to the database

  saveToDatabase(subscriberId, clickId, shortCode)

    .then(() => res.json({ status: "success" }))

    .catch(() => res.status(500).json({ status: "error" }));

});

function saveToDatabase(subscriberId, clickId, shortCode) {

  // Logic to save data in the database

  return new Promise((resolve, reject) => {

    // Insert into tbl_transaction

    const query = `INSERT INTO tbl_transaction (msisdn, clickId, shortCode, status, stage)

      VALUES ('${subscriberId}', '${clickId}', '${shortCode}', 'PENDING', 'INITIATED')`;

    // Execute database query

    db.execute(query, (err, result) => {

      if (err) reject(err);
    });
  });
}

```



```
    resolve(result);  
  });  
});  
}
```

```
app.listen(3000, () => console.log("Server running on port 3000"));
```

4. Web-to-SMS Functionality

Once the transaction is saved successfully, the customer is redirected to their default SMS app with the message "OK" pre-filled. This ensures the customer sends the required SMS to the short code to confirm their subscription.

```
const message = "OK";  
window.location.href = `sms:${shortCode}?body=${encodeURIComponent(message)}`;
```

5. Telco Callback & Validation

After the SMS is sent, the telco will call back your system with the subscription confirmation. You'll need to validate the phone number against the saved transaction.

1. **Receive the telco callback** with the phone number and SMS status.
2. **Validate the phone number** against the database.
3. **Update the transaction status** to "SUCCESS" if the SMS was sent successfully.

6. Marketer Postback Implementation

Once the telco confirms the SMS is valid, notify the marketer by making a GET request to their postback URL. The clickId that was saved in the database should be included in the postback.

```
const handlePostbackToMarketer = (clickId) => {  
  
  const url =  
    `http://m.mobplus.net/c/p/93a7719300a247d18ca8db99b8c91461?txid=${clickId}`;  
  
  fetch(url, { method: "POST" })  
    .then(response => {  
      if (response.ok) {  
        console.log("Marketer postback successful");  
      } else {  
        console.error("Marketer postback failed");  
      }  
    })  
    .catch(error => console.error("Postback error:", error));  
};
```

Summary of Implementation Steps:

1. **Create Database Table** to store transactions.
2. **Set up a Landing Page** to collect user phone numbers and clickId.
3. **Create a Backend API** to store the user's phone number and clickId.
4. **Implement Web-to-SMS Logic** to open the SMS app with a prefilled message.
5. **Handle Telco Callbacks** to validate the SMS and update the transaction status.

6. **Send a Postback Request** to the marketer once the SMS is confirmed.